

Railway Predictive Maintenance

Technical Documentation — Hackathon MVP

Anomaly Detection · RUL Estimation · Real-time Serving · Fleet Dashboard

Contents

- 0. System Architecture
 - 1. Beginner's Guide
 - 2. Model Evaluation & Benchmarks
 - 3. MRO / Asset-Management Integration Roadmap

Generated 2026-07-07

System Architecture

End-to-end data flow. The **offline** lane trains and evaluates the models; the **online** lane serves live telemetry in real time. The two lanes meet at the Model Store — models are trained once and loaded into the inference engine at startup, so the serving path never re-reads disk per event.

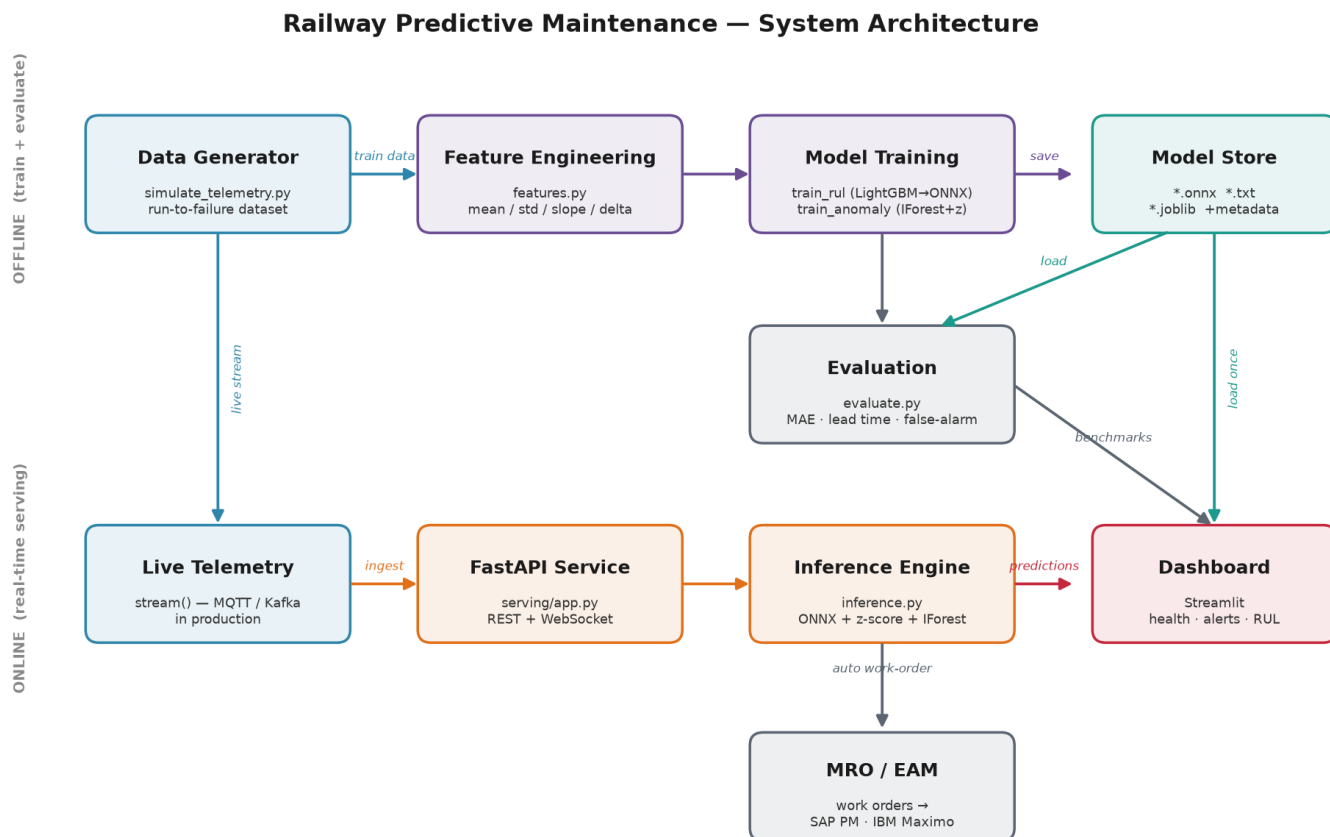


Figure 1 — Railway PdM system architecture (see docs/architecture_diagram.py to regenerate).

Beginner's Guide

A from-scratch explanation of this project for someone new to **machine learning**, **railway/aerospace maintenance**, **Python**, or **servers**. No prior background assumed. Read top to bottom, or jump to a section.

- [1. The problem in plain words](#)
- [2. Key ideas & vocabulary](#)
- [3. How the whole system fits together](#)
- [4. A tour of every file](#)
- [5. The machine-learning part, gently](#)
- [6. The "server" part, gently](#)
- [7. Run it yourself, step by step](#)
- [8. How we measure if it's any good](#)
- [9. FAQ](#)

1. The problem in plain words

A train (or aircraft) is full of parts that slowly wear out: bearings, brakes, motors. If a part fails while in service, you get an unplanned breakdown — expensive, disruptive, and sometimes dangerous.

There are two bad ways to handle this:

1. **Run to failure** — wait until it breaks. Cheapest until the day it strands a train on the mainline.
2. **Fixed schedule** — replace every part every N months no matter what. Safe, but wasteful: you throw away parts that still had plenty of life.

Predictive maintenance (PdM) is the smart third way: watch the part's sensors, notice when it *starts* to degrade, estimate how much life is left, and fix it *just before* it would fail — not too early, not too late.

That's exactly what this project does, for a simulated fleet of trains.

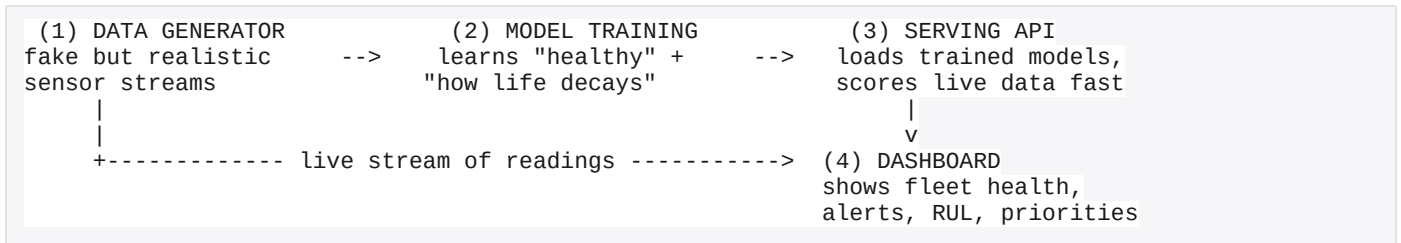
2. Key ideas & vocabulary

Term	Plain meaning
Asset	A physical part we monitor (a bogie bearing, a brake unit, a traction motor). "Bogie" = the wheel-and-axle assembly under a rail car. "Traction motor" = the electric motor that drives the wheels.
Telemetry / sensor data	Numbers streaming off the asset: vibration, temperature, pressure, current, etc. Like a fitness tracker for machines.
Channel	One sensor's stream (e.g. the "temperature" channel).
Operating cycle	One unit of use (think "one trip" or "one hour of running"). We measure age and remaining life in cycles.

Degradation	The gradual worsening of a part as it wears. Usually slow at first, then accelerating — a "knee curve".
Anomaly	Sensor behavior that no longer looks like a healthy part. The first sign of trouble.
RUL (Remaining Useful Life)	How many more cycles the part can run before it needs maintenance. The headline number a planner wants.
Baseline	What "healthy" looks like, learned from data. We flag deviations <i>from</i> the baseline.
Model	A mathematical function, fitted to past data, that makes a prediction (here: "is this an anomaly?" and "what's the RUL?").
Feature	An input number we feed the model. We don't feed raw sensors directly; we compute richer signals first (see §5).
False alarm	Crying wolf — flagging a healthy part as degrading. Too many and people stop trusting the system.
Lead time	How far <i>ahead</i> of failure we raise the alarm. More lead time = more time to plan a fix.

3. How the whole system fits together

Four moving pieces:



1. **Data generator** invents realistic sensor data (we don't have a real train fleet handy, so we simulate one — see §5 for why this is legitimate).
2. **Training** looks at lots of historical "run-to-failure" examples and learns two things: what healthy looks like, and how sensor trends map to remaining life.
3. **Serving API** is a small always-on program that holds the trained models in memory and answers "here's a new reading, what's the health/RUL?" in a couple of milliseconds.
4. **Dashboard** is the screen a maintenance planner actually looks at.

4. A tour of every file

```

Hackathon_Railway_PDM/
+-- data/
|   +-- simulate_telemetry.py  <- invents realistic sensor data + a live feed

```

```

+-- src/
|   +-- features.py           <- turns raw sensors into model inputs ("features")
|   +-- train_rul.py         <- trains the Remaining-Useful-Life predictor
|   +-- train_anomaly.py     <- trains the "is this healthy?" detector
|   +-- evaluate.py          <- measures how good the models are
+-- serving/
|   +-- inference.py         <- the fast scoring engine (models live here)
|   +-- app.py               <- the web API around the engine
|   +-- model_store/         <- saved trained models + benchmark report
+-- dashboard/
|   +-- app.py               <- the visual fleet dashboard
+-- docs/
|   +-- you are here
+-- requirements.txt         <- the Python libraries this project needs
+-- README.md               <- quick-start + architecture summary

```

You only ever *run* three things: the two training scripts, the serving API, and the dashboard. Everything else is supporting code they import.

5. The machine-learning part, gently

Why simulated data is OK here

Real run-to-failure datasets for rail are proprietary and scarce. So we generate synthetic data — but not random noise. Each simulated part follows a physically-motivated **degradation curve**: sensors stay flat and noisy while healthy, then trend upward (or downward) with an accelerating "knee" as the part approaches end-of-life. This is the same shape as NASA's well-known **C-MAPSS** turbofan degradation dataset. Because the degradation is *real* in the data, the models have to *learn* it rather than memorize noise — and the same code runs unchanged on real telemetry later (see the integration roadmap).

Features: why we don't feed raw sensors

A single temperature reading of 60°C tells you little. But "temperature has been *rising* for the last 20 cycles and its variability just doubled" is a strong degradation signal. So for each sensor we compute, over a sliding window of recent readings:

- **rolling mean** — the recent average (smooths out noise),
- **rolling standard deviation** — how jumpy it's been (wear often adds jitter),
- **rolling slope** — is it trending up or down, and how fast,
- **delta** — the change since the previous reading.

These derived numbers are the **features**. `src/features.py` computes them. The exact same feature math runs in training and in live serving — otherwise the model would see different inputs in the two settings ("train/serve skew").

Two models, two jobs

Anomaly detection ("is this still healthy?") — `src/train_anomaly.py`. We train an **Isolation Forest**, an algorithm that learns the shape of "normal" from healthy data *only* (it needs no examples of failures, which real fleets rarely have enough of). Anything that doesn't fit the normal shape gets a low score -> flagged. For the *real-time* path we also compute a cheaper **z-score** statistic (how many standard deviations each sensor is from its healthy baseline) so we can flag instantly on every event; the Isolation Forest runs as a slower, deeper second-opinion sweep. (See the README's "Why it's fast enough" section for the latency reasoning.)

RUL estimation ("how much life is left?") — `src/train_rul.py`. We train a **LightGBM** model — a "gradient-boosted decision tree" regressor. Think of it as a large committee of simple yes/no rules ("is the vibration slope above X? is the temperature variance above Y?") that together output a number: the predicted remaining cycles. It trains in seconds on a laptop and predicts in microseconds. We **cap** the target at 300 cycles because predicting "5000 cycles left" precisely is neither possible nor useful — planners only care once a part enters its final few hundred cycles.

What "training" actually does

Training = show the model many past examples where we know the right answer (the true RUL, or known-healthy data), and let it adjust its internal numbers until its predictions match. We then **save** the fitted model to disk (`serving/model_store/`). Serving just **loads** that saved model — no learning happens live.

6. The "server" part, gently

What is a server / API?

A **server** here is just a Python program that stays running and waits for requests. An **API** (Application Programming Interface) is the menu of requests it understands. Ours is built with **FastAPI**, a popular Python web framework.

You talk to it over HTTP — the same protocol your browser uses. Examples:

- `GET /health` -> "are you alive?" -> `{"status": "ok", ...}`
- `POST /ingest` -> "here's a sensor reading, score it" -> prediction back
- `GET /fleet/status` -> "give me every asset's latest health, worst first"
- `GET /metrics/model` -> "how accurate are your models?" (the benchmark report)

`GET` = "read something." `POST` = "send something / make something happen."

Why a server at all — why not just a script?

Because predictions have to happen **continuously and fast** as data streams in, potentially hundreds of readings per second across the fleet, forever. A server loads the (relatively slow-to-load) models **once** at startup and then answers each request in ~1 millisecond. Restarting a script per reading would be thousands of times slower.

`serving/inference.py` is the brain (loads models, scores a reading). `serving/app.py` wraps that brain in the web API and also runs a background loop that feeds it the simulated live fleet, so there's always something to look at.

The dashboard

`dashboard/app.py` uses **Streamlit**, a tool that turns a Python script into a web page with almost no web code. It periodically calls the server's API (`/fleet/status`, `/metrics/model`, `/workorders`) and draws tables and charts. It holds *no* intelligence itself — it's a window onto the server.

7. Run it yourself, step by step

You need **Python 3.10+** installed. Commands below are for Windows (PowerShell / Git Bash); on macOS/Linux replace `./venv/Scripts/` with `./venv/bin/`.

Step 1 — install the libraries (one time):

```
python -m venv .venv # make an isolated Python sandbox
./venv/Scripts/pip install -r requirements.txt
```

A "virtual environment" (`.venv`) keeps this project's libraries separate from the rest of your computer. `requirements.txt` lists what to install.

Step 2 — train the models (a couple of minutes):

```
./venv/Scripts/python -m src.train_rul # trains the RUL predictor
./venv/Scripts/python -m src.train_anomaly # trains the anomaly detector
```

This writes trained model files into `serving/model_store/`.

Step 3 — measure how good they are (optional but recommended):

```
./venv/Scripts/python -m src.evaluate
```

Prints benchmark tables and saves `evaluation_report.json`.

Step 4 — start the server (leave it running):

```
./venv/Scripts/python -m uvicorn serving.app:app --host 127.0.0.1 --port 8000
```

Visit `http://127.0.0.1:8000/docs` in a browser — FastAPI auto-generates an interactive page where you can click every endpoint. `uvicorn` is the program that actually runs the FastAPI app.

Step 5 — start the dashboard (in a second terminal):

```
./venv/Scripts/python -m streamlit run dashboard/app.py
```

It opens a browser tab showing the live fleet. Done.

8. How we measure if it's any good

We never trust a model on the data it trained on — that's like grading students on questions they've already seen. Instead `src/evaluate.py` builds a **fresh held-out fleet** (new random seed, assets the models never saw) and replays each part's whole life through the real serving path. Then it reports:

For RUL (regression accuracy):

- **MAE (Mean Absolute Error)** — on average, how many cycles off is the prediction? Lower is better. (*We get ~1.7 cycles overall.*)
- **RMSE** — like MAE but punishes big misses more.
- **R²** — fraction of the variation the model explains, 0–1. (*We get ~0.97.*)
- **Error by RUL band** — accuracy specifically when the part is *close* to failure, which is what matters most for planning.

For anomaly detection:

- **Detection lead time** — how many cycles of early warning we get before end-of-life. More = more time to act. (*Median ~1390 cycles.*)
- **False-alarm rate** — fraction of genuinely-healthy readings we wrongly flag. Lower = more trustworthy. (*~1%.*)
- **Detection rate** — of all failing parts, how many did we catch in time? (*100% in the benchmark.*)

- **Precision / Recall** — of our alarms, how many were real (precision); of the real problems, how many we caught (recall).

These are the numbers in [docs/EVALUATION.md](#) and on the dashboard's "Model quality benchmarks" panel.

9. FAQ

Q: Is the data real? No — it's simulated, but with realistic physics-based degradation (\$5). The code path is built so that swapping in real sensor streams changes only the ingestion adapter, not the models (see the integration roadmap).

Q: What's a "cycle"? Our unit of usage/age. In a real deployment it might be an operating hour, a trip, or a duty cycle — whatever the maintenance schedule is denominated in.

Q: Why two anomaly methods (z-score and Isolation Forest)? Speed vs. depth. The z-score is instant and runs on every event; the Isolation Forest is a richer multivariate check that's too slow per-event, so it runs as a periodic batched sweep. Together: fast alarms + deep second opinion.

Q: Why cap RUL at 300 cycles? Because predicting "3,412 cycles left" precisely is impossible and pointless. Planners act in the final stretch, so we focus the model's accuracy there.

Q: Where do I see everything working? The Streamlit dashboard (Step 5) — fleet table, alerts, RUL charts, work orders, and the benchmark panel — all live.

Q: I'm an ML person — where's the interesting engineering? The train/serve feature-parity design in `features.py` (batch rolling ops for training, O(1) incremental buffers for serving) and the two-tier anomaly architecture that keeps p99 latency ~3ms. See the README.

Model Evaluation & Benchmarks

All numbers below are produced by `src/evaluate.py` and saved to `serving/model_store/evaluation_report.json`. They are also served live at `GET /metrics/model` and shown on the dashboard's "Model quality benchmarks" panel.

Regenerate any time with:

```
python -m src.evaluate
```

Methodology (why these numbers are trustworthy)

- **Held-out test fleet.** We generate a *fresh* set of run-to-failure assets with a different random seed (2024) from the one used for training (42). The models have never seen these assets.
- **Real serving path.** Each asset's entire life is replayed cycle-by-cycle through the exact production `InferenceEngine.process_event` — the same code that serves live traffic. So the metrics reflect real behaviour, including the incremental feature buffers, not an idealized offline pipeline.
- **12 assets per type × 3 asset types = 36 unseen assets**, ~141,600 scored cycles total.

1. RUL accuracy benchmarks

Error is measured in **operating cycles** against the capped RUL target (cap = 300 cycles — see note below). Lower MAE/RMSE and higher R² are better.

Asset type	MAE (cycles)	RMSE (cycles)	R ²	n (cycles)
bogie_bearing	1.26	6.55	0.978	52,031
brake_unit	3.45	13.37	0.950	27,514
traction_motor	1.34	7.12	0.970	62,055
overall	1.72	8.53	0.967	141,600

Accuracy near failure (what actually matters)

A planner cares most about accuracy when a part is *close* to failure. Error broken down by how much true life remains:

RUL band (cycles)	bogie MAE	brake MAE	motor MAE
0–50 (imminent)	4.56	4.17	6.86
50–150	11.69	17.40	13.67
150–300	24.06	34.80	28.33

300+ (healthy tail)	0.12	0.37	0.20
---------------------	------	------	------

Reading this table: - In the **final 50 cycles**, predictions are within ~4–7 cycles — tight enough to schedule an intervention confidently. - The **150–300 band** has the largest error: this is the onset of degradation, where the future is genuinely most uncertain. Error shrinking as failure approaches is exactly the desirable behaviour. - The **300+ tail** is near-zero because those cycles are all correctly predicted at the cap.

Why cap RUL at 300 cycles? Early in a healthy part's life the true RUL can be thousands of cycles. Predicting that precisely is impossible and operationally useless — nobody schedules maintenance 4,000 cycles out. So the model is trained (and scored) to say ">= 300, plenty of life" until a part enters its actionable final window, then to sharpen. This mirrors the standard piecewise-linear RUL target used on NASA C-MAPSS.

2. Anomaly detection metrics (incl. detection lead time & false-alarm rate)

Asset type	Detection rate	Median lead time (cycles)	Mean lead time	False-alarm rate	Precision	Recall	F1
bogie_bearing	100%	1,707	1,687	1.03%	0.984	0.931	0.957
brake_unit	100%	819	863	0.90%	0.985	0.868	0.923
traction_motor	100%	1,977	1,972	1.14%	0.981	0.898	0.938
overall	100%	1,390	1,507	1.05%	0.983	0.904	0.942

What each metric means

- **Detection rate** — fraction of failing assets flagged (with a sustained alarm) *before* end-of-life. **100%** — every failure was caught in the benchmark.
- **Detection lead time** — operating cycles of early warning between the first sustained alarm and end-of-life. A **median of ~1,390 cycles** means we typically raise the flag well before failure, leaving ample planning runway. (Lead time varies by asset because each type wears over a different lifespan.)
- **False-alarm rate** — fraction of genuinely-healthy readings (first 60% of life) that were wrongly flagged. **~1%** — low enough to stay trustworthy; the threshold is deliberately set at the 99th percentile of healthy scores.
- **Precision** — of all alarm-cycles, the fraction that occurred while the asset was actually degrading (not healthy). **0.98.**
- **Recall** — of all degrading-cycles, the fraction we flagged. **0.90.**

How "detection" is defined

- **Sustained alarm (debounce):** we require **3 consecutive** flagged cycles before declaring a detection, so a single noisy spike doesn't count. This is standard practice to suppress jitter.
- **Positive class = "degrading":** for precision/recall, a cycle is labelled positive if the asset is past its healthy window (life fraction ≥ 0.6 — the same boundary the detector was trained on). Scoring instead against only the last 10% of life would unfairly penalize correct *early* warnings as false positives.

3. Inference latency (real-time SLA)

Not part of the offline report, but measured live via GET `/metrics/latency` while the demo fleet streams:

Percentile	Latency per event
p50	~0.6 ms
p95	~1.8 ms
p99	~3 ms

This includes **both** the RUL prediction and the anomaly check for one reading. See the README's "Why it's fast enough for production" section for the engineering behind these numbers (ONNX Runtime, O(1) incremental features, two-tier anomaly detection).

Reproducing / tuning

Key knobs in `src/evaluate.py`:

Constant	Meaning	Default
TEST_SEED	RNG seed for the held-out fleet	2024
N_TEST_ASSETS_PER_TYPE	assets per type in the test fleet	12
IMMINENT_FRACTION	last-fraction-of-life = "imminent" window	0.10
DEBOUNCE	consecutive flags required to declare detection	3
RUL_BANDS	bands for the near-failure accuracy breakdown	see file

Raising DEBOUNCE reduces false alarms but slightly shortens lead time; lowering the anomaly threshold (in `src/train_anomaly.py`) does the opposite. This tradeoff is the classic sensitivity vs. specificity dial a real deployment would tune to the customer's tolerance for false alarms.

MRO / Asset-Management Integration Roadmap

Goal: connect this predictive-maintenance engine to the systems a railway already runs — the MRO (Maintenance, Repair & Overhaul) and EAM (Enterprise Asset Management) platforms — so that a predicted failure automatically turns into a scheduled, parts-planned, costed work order, with no analyst copy-pasting between screens.

New to these acronyms? See the glossary at the bottom.

Where we are today (MVP)

The system already produces the *content* an MRO system needs and exposes it over a clean HTTP API:

Capability	Endpoint	Status
Per-asset health, RUL, anomaly flag	GET <code>/fleet/status</code> , GET <code>/assets/{id}</code>	[OK] built
Prioritized ranking (what to fix first)	GET <code>/fleet/status</code> (sorted)	[OK] built
Work-order creation	POST <code>/workorders</code>	[OK] stub (in-memory)
Work-order listing	GET <code>/workorders</code>	[OK] stub (in-memory)
Auto work-order on urgent asset	internal <code>_maybe_auto_workorder</code>	[OK] built
Live push feed	WS <code>/ws/stream</code>	[OK] built

"Stub" means the work order is created and tracked in memory to prove the workflow end-to-end, but it is **not yet written into a real MRO system of record**. That connection is what this roadmap covers.

The important design property: the *prediction path never changes* across these phases. Integration is always an **adapter** bolted onto the existing API — we are adding output destinations, not re-architecting the models.

Phase 1 — Read-only integration (weeks 1–3)

Objective: MRO/EAM systems and dashboards can *read* our predictions. No writes into external systems yet — lowest risk, immediate value.

- **Stable REST contract + OpenAPI spec.** FastAPI already auto-publishes an OpenAPI schema at `/openapi.json` and interactive docs at `/docs`. Version it (`/v1/...`) and freeze the response shapes.
- **Asset-ID mapping table.** Our `asset_id` (e.g. `train_03_bogie_bearing_0`) must map to the customer's canonical equipment number (SAP *Equipment*, Maximo *Asset* record). Add a small mapping service / DB table: `asset_id <-> external_equipment_id <-> functional_location`.
- **Authentication.** Put the API behind an API gateway with OAuth2 client credentials or mutual-TLS; add per-consumer rate limits.
- **Historian read (optional).** If the customer stores sensor data in a historian (OSIsoft PI, Aveva, InfluxDB), add a reader so we ingest *their* telemetry instead of the simulator (see Phase 3).

Deliverable: the customer's BI tools and our dashboard both read live health/RUL from a documented, authenticated `/v1/fleet/status`.

Phase 2 — Work-order write-back (weeks 4–8)

Objective: an urgent prediction automatically becomes a real work notification/order in the customer's MRO system.

- **Replace the in-memory work-order store with an MRO adapter interface:**

```
class WorkOrderSink(Protocol):  
    def create(self, wo: WorkOrder) -> str: ... # returns external WO id  
    def update_status(self, ext_id: str, status: str) -> None: ...
```

Ship concrete implementations: - **SAP PM / SAP S/4HANA Asset Management** — create a *Notification* (type M2) and/or *Maintenance Order* via SAP's OData / BAPI (`BAPI_ALM_NOTIF_CREATE`, `BAPI_ALM_ORDER_MAINTAIN`). - **IBM Maximo** — create a *Work Order* through the Maximo REST/OSLC API (`/maximo/api/os/mxwo`). - **Generic webhook** — POST our work-order JSON to any endpoint, for systems without a native connector.

- **De-duplication & lifecycle.** Only one open work order per asset per root cause (already enforced in the stub via `already_open`). Subscribe to status callbacks so closing the WO in SAP/Maximo flows back to us.
- **Human-in-the-loop gate.** Configurable: auto-create vs. "propose and wait for planner approval." Railways will start with propose-only.
- **Enrichment.** Attach to each work order: predicted RUL, anomaly evidence (which sensor channel deviated — we already compute `worst_sensor`), recommended lead time, and a link back to the asset's dashboard view.

Deliverable: crossing the urgent-priority threshold creates a real, de-duplicated, evidence-rich notification in SAP PM or Maximo.

Phase 3 — Ingestion from real telemetry (weeks 6–10, parallel)

Objective: stop using the simulator; consume the customer's real sensor streams.

- **Streaming ingestion.** Swap `LiveTelemetrySimulator` for a Kafka / MQTT / AMQP consumer. Each incoming message is normalized to the same `reading dict` and passed to the unchanged `InferenceEngine.process_event`.
- **Schema mapping & units.** Map the customer's channel names/units to our canonical sensor names per asset type; validate ranges; handle missing channels gracefully.
- **Backfill / historian replay.** Re-train on the customer's historical run-to-failure data as it becomes available, replacing the synthetic set.
- **Edge option.** For trains with poor connectivity, the same lightweight ONNX model can run on an onboard edge gateway and only push predictions/alarms upstream.

Deliverable: predictions are driven by real onboard sensors, models retrained on real degradation history.

Phase 4 — Closed-loop planning & scheduling (weeks 10–16)

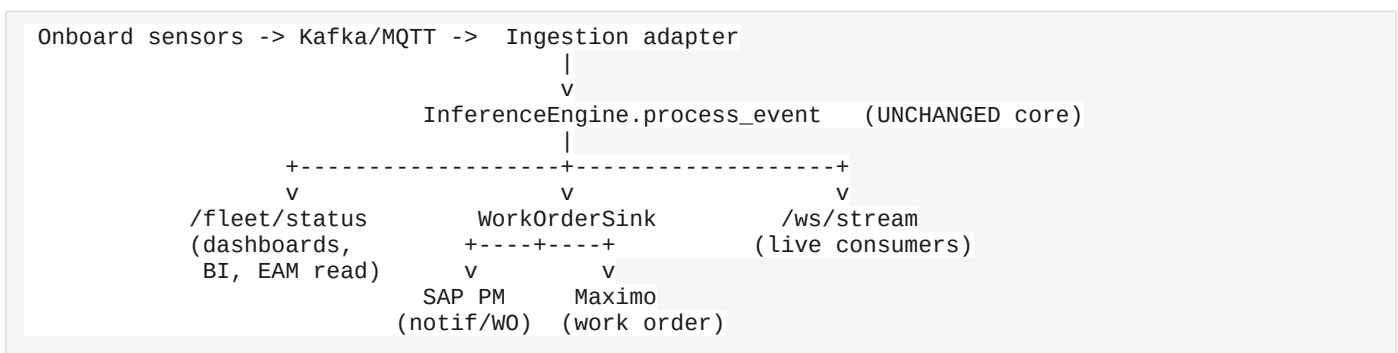
Objective: predictions influence *when and how* maintenance is scheduled, not just *that* it's needed.

- **Spare-parts / inventory check.** Before proposing a date, query the MRO inventory module for part availability; flag long-lead parts early.

- **Scheduling optimization.** Feed RUL windows into the depot scheduler so interventions cluster efficiently (group assets on the same train / same depot visit) while staying inside each asset's safe RUL window.
- **Feedback loop for model improvement.** When a work order closes, capture the actual findings (was the part really degraded? was it a false alarm?). This labelled outcome data is fed back to periodically retrain and to track live precision / false-alarm rate against the offline benchmarks.
- **Regulatory & audit trail.** Log every prediction->notification->work-order chain for safety audits (important in rail).

Deliverable: a measurable reduction in unplanned downtime, with a data-driven feedback loop continuously improving the models.

Integration architecture (target state)



Everything left of `InferenceEngine` is *ingestion adapters*; everything right of it is *output adapters*. The models and scoring never change — which is exactly why integration is low-risk.

Standards & protocols we align to

- **REST + OpenAPI** for synchronous reads/writes (already emitted by FastAPI).
- **OData / BAPI** for SAP PM; **REST/OSLC** for IBM Maximo.
- **Kafka / MQTT / AMQP** for telemetry ingestion.
- **ISO 14224** (reliability & maintenance data for equipment) and **MIMOSA OSA-EAI / OSA-CBM** as the vocabulary for condition-based maintenance interchange — worth adopting for asset/condition data models so we speak the same language as enterprise EAM tools.

Glossary (for newcomers)

- **MRO** — *Maintenance, Repair & Overhaul*. The systems and processes that keep physical assets serviceable.
- **EAM** — *Enterprise Asset Management*. Software (SAP PM, IBM Maximo) that tracks assets, work orders, spare parts, and maintenance history.
- **Work order / Notification** — the formal record telling a technician to inspect or repair a specific asset. A "notification" (SAP) often precedes a full "work order."
- **Functional location / Equipment number** — how enterprise systems uniquely identify a physical asset and its place in the fleet hierarchy.
- **Historian** — a time-series database purpose-built for industrial sensor data (e.g. OSIsoft PI).

- **Edge gateway** — a small computer on the vehicle that can run the model locally when the network is unreliable.